There's one instinctive and imperative way to solve this: by adding even more state and going further down the rabbit hole!

```
numberSold = 0
calculateNumberSold = (salePrice) ->
  numberSold = 50 + 20/10 * (200 - salePrice)

calculateRevenue = (salePrice, callback) ->
  callback numberSold * salePrice

revenueBetween = (start, finish, callback) ->
  totals = []
  receivedResponses = 0
  expectedResponses = 0
  for price in [start..finish]
    calculateNumberSold price
    expectedResponses++
    addToTotals = (result) ->
      totals.push result
      receivedResponses++
      if receivedResponses == expectedResponses
        callback totals
    calculateRevenue price, addToTotals
```

Add state by keeping variables for expected and received responses.

Does it work now? Sadly, no. The order of the values will depend on the order in which the callbacks return. Sure, you'll get the revenue values—just not in the order you want!

The problem becomes even worse if some other part of the program changes the value of `numberSold` while an asynchronous part of your program is waiting. Remember, `numberSold` is shared state for several functions.

Can `revenueBetween` get any more intertwined? Indeed it can. Suppose Mr. Tuse modifies the function to also log a message, and for some inexplicable reason changes the value of `numberSold` at the same time: